

Foundation Course YAML Front Matter Schema

<https://gemini.google.com/app/5a0e6b5c41b54ec1>

User prompt: Okay, I'm on some kind of final touches, you know, um, I reworked the YouTube array of objects, um, where it's just essentially a video object with a source, which hopefully I can pull more data from uh, YouTube if I need it and I if I were stuck with an alt, you know, I can put a little extra stuff or data in there. Podcast episode I'm probably going to be using Spotify and I imagine the same thing. I might be able to use an API to get more information rather than loading that on handwritten data. But then I have these things which is like block talk, uh, content, homeschooling supporting content, board game content, children's content. These are kind of loose taxonomical categories and I just kind of want to like, you know, I want some help coming up with a reliable object schema for these general categories. These objects

Response: First off, keeping media tags like youtube and podcast lean with just src and alt is a fantastic call. Fetching dynamic metadata (titles, duration, thumbnails) via build-time oEmbed or API calls keeps your handwritten burden virtually zero.

For these nascent taxonomical categories (block_talk_content, homeschooling_supporting_content, board_game_content, childrens_content), the goal is to create a **single, unified item schema** that works across all four arrays. You don't want four different object shapes; you want **one flexible "resource item" pattern** so your brain only has to remember one set of tag attributes when writing in your sketchbook.

The Universal Resource Item Schema

Whether it's a board game reference, a block talk point, or a children's activity, every entry in these collections typically needs four core properties:

1. **title** (string, required): The name of the game, activity, lesson, or topic.
2. **url** (string/URL, optional): External reference link (e.g., BoardGameGeek link, Amazon item, curriculum PDF, or reference article).
3. **order** (number, optional, defaults to document position): Keeps our standard non-linear decimal insertion capability (1, 1.25, 1.5) alive across all lists.
4. **notes** (string, optional / inner body): Brief contextual advice, setup rules, teaching guidance, or commentary.

How It Translates to Zod & TypeScript

We define one generic ResourceContentSchema and re-use it for all four block categories:

```
import { z } from 'zod';

// Generic item schema for all custom content collections
export const ResourceContentSchema = z.object({
  order: z.number().default(1),
  title: z.string(),
  url: z.string().url().optional(),
  notes: z.string().optional(), // Can be set via attribute OR tag body text
});

export type ResourceContent = z.infer<typeof ResourceContentSchema>;

// Top-level arrays inside your Base Document Schema
export const BaseContentCategoriesSchema = z.object({
  block_talk_content: z.array(ResourceContentSchema).default([]),
  homeschooling_supporting_content: z.array(ResourceContentSchema).default([]),
  board_game_content: z.array(ResourceContentSchema).default([]),
  childrens_content: z.array(ResourceContentSchema).default([]),
});
```

Handwriting Example Across Categories

By keeping the schema uniform, your tag syntax remains completely predictable in your sketchbook.

1. Board Game Content (board_game_content)

```
<!-- block:board_game_content -->
<!-- item:game order="1" title="Catan" url="https://boardgamegeek.com/boardgame/13/catan" -->
Use basic trading loops to demonstrate resource management and barter dynamics.
<!-- /item:game -->
<!-- /block:board_game_content -->
```

2. Homeschooling Supporting Content (homeschooling_supporting_content)

```
<!-- block:homeschooling_supporting_content -->
<!-- item:resource order="1" title="Clemson Extension Soil Analysis Sheet" url="https://example.com/soil.pdf" -->
Print out copy for field testing nitrogen levels in raised beds.
<!-- /item:resource -->
<!-- /block:homeschooling_supporting_content -->
```

3. Children's Content (childrens_content)

```
<!-- block:childrens_content -->
<!-- item:activity order="1" title="Cardboard Timber Framing" -->
Hands-on building exercise translating triangular shed geometry to mini models.
<!-- /item:activity -->
<!-- /block:childrens_content -->
```

Parser Behavior: Alias Matching

Notice in the handwritten examples above that inside `<!-- block:board_game_content -->`, you might naturally write `<!-- item:game -->` or `<!-- item:resource -->`.

In your Pass 1 parser logic, you can simply tell the block parser: "Any `item:*` tag inside a known block array gets parsed into the `ResourceContentSchema`."

This means whether you write `item:game`, `item:activity`, or just `item:content`, the parser normalizes it straight into `{ title, url, order, notes }`. You get total cognitive freedom while handwriting, and strict type-safety in 11ty.

User prompt: love your resource content schema concept. It really does free me up. And so let's let's touch on some of your four ideas. I like the idea of title. Um, I like the idea of notes. Um, I think I don't know if I like the idea of order because some of these are going to be so broad. I wouldn't even know how to conceivably juggle that. Uh, but I love I do like the idea of adding based on what you're saying, um, uh, an array of image URLs, um, an array of audio URLs or an array of uh, video URLs, which could all be YouTube. Um, so I think I want to add those things. I don't know that I want order, but it doesn't hurt, right? I guess I can make order optional and maybe for certain type, it's useful. Uh, and so I don't know that I want to get rid of it. I love the idea of the resource content schema and uh, yeah.

Response: That is a great call. Making `order` optional is the best of both worlds: for a sequential mind movie or outline chain, order is essential, but for broad taxonomical resources like board games or children's activities, forcing yourself to assign numbers while writing in your sketchbook just adds mental friction.

If `order` is missing on a resource item, your parser simply retains the document's natural writing sequence.

Adding pluralized array attributes for media (`image_urls`, `audio_urls`, `video_urls`) turns `ResourceContentSchema` into a true Swiss Army knife. You can attach a YouTube clip, a Spotify track, or sketchbook photo links straight inside the tag attributes using simple comma-separated lists.

The Universal ResourceContentSchema

Here is how the refined Zod schema looks with optional ordering and pre-processed media arrays:

```
import { z } from 'zod';

// Helper to preprocess comma-separated attribute strings into typed string arrays
const commaSeparatedUrls = z.preprocess((val) => {
  if (typeof val === 'string') {
    return val.split(',').map((url) => url.trim()).filter(Boolean);
  }
  return val;
}, z.array(z.string().url()).default([]));

export const ResourceContentSchema = z.object({
  title: z.string(),

  // Optional order: if omitted, pipeline uses natural sketchbook document order
  order: z.number().optional(),

  // Optional URL link (e.g., BoardGameGeek, documentation, curriculum link)
  url: z.string().url().optional(),

  // Pluralized media arrays (parsed from comma-separated string attributes)
  image_urls: commaSeparatedUrls,
  audio_urls: commaSeparatedUrls,
  video_urls: commaSeparatedUrls,

  // Rich text narrative or commentary (from tag body or attribute)
  notes: z.string().optional(),
});

export type ResourceContent = z.infer<typeof ResourceContentSchema>;
```

Handwriting Examples in Your Sketchbook

Because all attributes use the same familiar syntax, writing across different categories in your sketchbook is completely effortless.

1. Board Game Content (Simple text + reference URL)

```
<!-- block:board_game_content -->
<!-- item:game title="Catan" url="https://boardgamegeek.com/boardgame/13/catan" -->
Great for introducing resource trading loops and land negotiation dynamics.
<!-- /item:game -->
<!-- /block:board_game_content -->
```

2. Homeschooling Content (With images and video references)

```
<!-- block:homeschooling_supporting_content -->
<!-- item:resource title="Soil Microbe Identification" video_urls="https://youtube.com/watch?v=123" image_urls
Compare soil sample slides under microscope to reference charts.
<!-- /item:resource -->
<!-- /block:homeschooling_supporting_content -->
```

3. Block Talk Content (Ordered sequence when you actually want order)

```
<!-- block:block_talk_content -->
<!-- item:talk order="1" title="Permaculture Ground Covers" audio_urls="https://open.spotify.com/track/456" --
Key discussion points for winter cover crop selection and soil building.
<!-- /item:talk -->
<!-- /block:block_talk_content -->
```

Why This Schema Gives You Total Freedom

- 1. **Zero Guilt Over Ordering:** If you're just brain-dumping board games or homeschooling links, leave `order` out completely.
- 2. **Rich Media Ready:** You can attach multiple YouTube videos or images to a single resource entry without creating complex inner child tags.
- 3. **One Tag Mindset:** You only have to remember one core pattern (`title`, `url`, `notes`, plus media arrays) across all your auxiliary content blocks.

User prompt: Sweet. Could you just go ahead and list out um that kind of universal object schema the in in in markdown that way I could copy and paste it into that other format.

Response: Here is the unified ResourceContent object schema formatted in Markdown, complete with TypeScript type definitions, Zod validation, and example HTML comment tags. You can copy and paste this directly into your documentation, issues, or Copilot instructions.

Universal Resource Content Schema

A unified schema designed for auxiliary taxonomical content categories (`block_talk_content`, `homeschooling_supporting_content`, `board_game_content`, `childrens_content`). It supports optional ordering, rich markdown notes, and pre-processed comma-separated media lists.

1. Data Structure Overview

Property	Type	Required / Optional	Description
title	string	Required	The primary display name of the resource, activity, game, or topic.
order	number	Optional	Numeric index for custom sorting (supports floats like 1.5). Defaults to document position if omitted.
url	string (URL)	Optional	Primary external reference link (e.g., BoardGameGeek page, PDF guide, article).
image_urls	array<string>	Optional	Array of image URLs parsed from a comma-separated string attribute.
audio_urls	array<string>	Optional	Array of audio URLs (Spotify, MP3) parsed from a comma-separated string attribute.
video_urls	array<string>	Optional	Array of video URLs (YouTube, Vimeo) parsed from a comma-separated string attribute.
notes	string	Optional	Narrative description, rules, teaching guidance, or commentary (captured via attribute or inner tag body).

2. TypeScript & Zod Implementation

```
import { z } from 'zod';
```

```

/**
 * Preprocessor helper: splits comma-separated URL strings into clean string arrays
 */
const commaSeparatedUrls = z.preprocess((val) => {
  if (typeof val === 'string') {
    return val.split(',').map((url) => url.trim()).filter(Boolean);
  }
  return val;
}, z.array(z.string().url()).default([]));

/**
 * Universal Resource Content Schema
 */
export const ResourceContentSchema = z.object({
  title: z.string(),
  order: z.number().optional(),
  url: z.string().url().optional(),
  image_urls: commaSeparatedUrls,
  audio_urls: commaSeparatedUrls,
  video_urls: commaSeparatedUrls,
  notes: z.string().optional(),
});

export type ResourceContent = z.infer<typeof ResourceContentSchema>;

/**
 * Base Content Categories Schema
 */
export const BaseContentCategoriesSchema = z.object({
  block_talk_content: z.array(ResourceContentSchema).default([]),
  homeschooling_supporting_content: z.array(ResourceContentSchema).default([]),
  board_game_content: z.array(ResourceContentSchema).default([]),
  childrens_content: z.array(ResourceContentSchema).default([]),
});

export type BaseContentCategories = z.infer<typeof BaseContentCategoriesSchema>;

```

3. HTML Comment Tag Format (Handwriting / Issue Body)

Basic Resource (Title + Reference Link + Inner Notes)

```

<!-- block:board_game_content -->
<!-- item:game title="Catan" url="https://boardgamegeek.com/boardgame/13/catan" -->
Great for introducing resource trading loops and land negotiation dynamics.
<!-- /item:game -->
<!-- /block:board_game_content -->

```

Media-Rich Resource (Images & Video references)

```

<!-- block:homeschooling_supporting_content -->
<!-- item:resource title="Soil Microbe Identification" video_urls="https://youtube.com/watch?v=123" image_urls
Compare soil sample slides under microscope to reference charts.
<!-- /item:resource -->
<!-- /block:homeschooling_supporting_content -->

```

Explicitly Ordered Resource

```

<!-- block:block_talk_content -->
<!-- item:talk order="1" title="Permaculture Ground Covers" audio_urls="https://open.spotify.com/track/456" -->
Key discussion points for winter cover crop selection and soil building.
<!-- /item:talk -->
<!-- /block:block_talk_content -->

```

User prompt: Hey, so you're doing such an awesome job helping me. It's revolutionizing the way I write and it's helping me. I'm changing a lot of the names for the properties just because I'm using this great tool you gave me just kind of fine tuning it. Um, and this might be pedantic, but I'm wondering, is this a good way to describe a kind of dictionary? It's going to be a concept lexicon, so it won't exactly boil down to just words uh taxonomically, but would you call it, should I give it the property name or should I give it the property property entry? Is that better for something like a lexicon or dictionary?

Response: Between name and entry, **entry is significantly better** for a lexicon or dictionary structure—but **term** or **concept** might actually be the absolute sweet spot depending on how you write them in your sketchbook.

Here is a breakdown of why entry beats name, along with a few options to consider for your tag attributes.

Why entry Beats name

- 1. **Precision:** name usually describes a person, place, or entity (e.g., author_name, user_name). For a dictionary/lexicon, the headline item is a headword or entry name.
- 2. **Domain Fit:** Lexicography formally calls each item in a lexicon a "lexical entry." Using entry="Permaculture" directly mirrors real-world dictionary architecture.
- 3. **Harmonizes with concept_term:** Since your item object is named concept_term, writing entry="..." feels natural (<!-- item:concept_term entry="Hugelkultur" -->).

Comparing Your Naming Options for Handwriting

Attribute Property	Handwriting Tag Example	Pros & Cons
entry (Recommended)	<!-- item:concept_term entry="Nikkud" -->	Pro: Clean, standard lexicography term. Con: Slightly generic if separated from context.
term (Strong Contender)	<!-- item:concept_term term="Nikkud" -->	Pro: Very short, punchy, and self-documenting for a lexicon. Con: Repeats the object name (concept_term.term).
name	<!-- item:concept_term name="Nikkud" -->	Pro: Generic string identifier. Con: Sounds like an entity or person rather than a dictionary term.

Recommended Zod Schema Layout

If you go with entry (or term), here is how your updated schema and tag syntax align cleanly:

```
import { z } from 'zod';

export const ConceptTermSchema = z.object({
  // 'entry' (or 'term') represents the word/concept being defined
  entry: z.string(),

  // Definition narrative (can live in tag attribute OR inner tag body)
  definition: z.string().default(""),

  // References / Cross-links (parsed from comma-separated string attribute)
  references: z.preprocess((val) => {
    if (typeof val === 'string') {
      return val.split(',').map((r) => r.trim()).filter(Boolean);
    }
    return val;
  }, z.array(z.string())).default([]),
});

export type ConceptTerm = z.infer<typeof ConceptTermSchema>;
```

How It Looks in Your Sketchbook

```
<!-- block:concept_lexicon -->

<!-- item:concept_term entry="Permaculture" references="Base 02, Base 05" -->
An ethical and practical design framework using systemic, ecological principles to create sustainable human en
<!-- /item:concept_term -->

<!-- /block:concept_lexicon -->
```

Using entry gives you exact domain alignment for a dictionary/lexicon, while keeping your handwritten tag attributes concise and expressive!

User prompt: This is a weird question. I know that basically whatever I write in my sketchbook will apply to the property text. Would somebody think entry includes the text or would somebody think entry includes the taxonomic name and text would just be the text of the entry.

Response: Not a weird question at all—in taxonomy and system architecture, this distinction is huge!

In domain modeling, **entry almost always refers to the entire record as a whole**, not just the headline word.

If someone hears "a dictionary entry," they picture the complete package: the word, the pronunciation, the definition, and the etymology together.

How People (and Parsers) Interpret These Terms

- **entry (The Whole Container):** In database design, an "entry" is the entire row or object. If you use entry as a field *inside* the object alongside text or definition, people might wonder: "Wait, is *entry* the word, or is *entry* the full body of text?"
- **term or headword (The Label):** This explicitly means *just* the title, concept name, or word being defined. There is zero ambiguity here.
- **text / definition (The Content):** This is unambiguously the body narrative explaining the concept.

Clarifying the Property Names

To avoid any confusion about what belongs where, using **term** (or **headword**) alongside **text** (or **definition**) creates a crystal-clear distinction:

Property Name	Role in Your Lexicon	What It Holds
term (or headword)	The Taxonomic Name	The specific word, phrase, or concept title (e.g., "Hugelkultur").
text (or definition)	The Body Content	The handwritten explanatory narrative inside the tag body.

How It Reads in Your Sketchbook

By using **term** for the label and letting the inner tag body naturally map to **text**, the boundary is unmistakable:

```
<!-- block:concept_lexicon -->

<!-- item:concept_term term="Hugelkultur" references="Base 04, Base 08" -->
A composting process employing raised planting beds built on top of decaying wood debris and plant materials.
<!-- /item:concept_term -->

<!-- /block:concept_lexicon -->
```

Here, **term** clearly isolates the taxonomic title ("Hugelkultur"), while the inner body provides the **text** of the entry.

User prompt: Okay, so I really like your ideas. I just kind of change it up. I'm going to let the entry be the name of the object, the entire object, based on what you said. I'm going to let term be you know, the kind of name, the taxonomical name. But I think I'm going to be a little more specific with this particular um property on the base schema. And I'll use definition in the place of text. It just helps my brain with specificity. So, I like it. I don't mind writing definition.

Response: That adjustment lands in the exact right spot. Renaming the parent object to `concept_entry` aligns perfectly with standard lexicography, while using **term** for the label and **definition** for the body creates total clarity in your schema.

Mapping the tag's inner body directly to **definition** in your parser means you won't even have to write out the word "definition" as an attribute while journaling—your handwritten narrative simply sits inside the tag.

Zod Schema Definition

Here is how your updated image layout maps directly into TypeScript and Zod:

```
import { z } from 'zod';

export const ConceptEntrySchema = z.object({
  // The taxonomical word or concept title
  term: z.string(),

  // The core narrative definition (captured from inner body or attribute)
  definition: z.string().default(""),

  // Optional diagram or visual illustration URL
  image: z.string().url().optional(),

  // Array of cross-references, parsed from comma-separated attribute string
  references: z.preprocess((val) => {
    if (typeof val === 'string') {
```

```

    return val.split(',').map((r) => r.trim()).filter(Boolean);
  }
  return val;
}, z.array(z.string())).default([]),
});

```

```
export type ConceptEntry = z.infer<typeof ConceptEntrySchema>;
```

How It Looks in Your Sketchbook

```
<!-- block:concept_lexicon -->
```

```

<!-- item:concept_entry term="Hugelkultur" image="https://example.com/bed-diagram.png" references="Base 04, Ba
A soil-building and water-retention technique using raised planting beds constructed over decaying log mounds
<!-- /item:concept_entry -->

```

```
<!-- /block:concept_lexicon -->
```

Why This Hierarchy Works So Well

1. **concept_entry**: Clearly signals that this entire block is a complete record in your lexicon.
2. **term**: Unambiguously isolated in the attributes as the headline name.
3. **definition**: Naturally holds the rich prose sitting inside the tag body without requiring extra markup.
4. **image & references**: Cleanly store visual and taxonomical cross-links right on the entry level.

User prompt: # BASE SCHEMA - title: string - number: integer <0-101> - short_description: string - long_description: string - google_docs_url: url - google_slides_url: url - images: array - image: object - src: url - alt: string - main_image: boolean - videos: array - video: object - src: url - alt: string - main_video: boolean - part_parent: integer <1-5> - scripture_list: array - scripture: object - ref: string - essential: boolean - order: number - teacher_notes: string - text: string - outline_chain: array - outline_link: object - order: number - notes: string <optional> - text: string - outline_sublink: object - order: number - order_parent: number <optional> - notes: string <optional> - text: string - concept_lexicon: array - concept_entry: object - term: string - definition: string - image - references: array <strings of base numbers> - practices: array - practice: object - name: string - description: - steps: array - step: object - number: integer - instructions: string - images: array - image: object <defined as above> - booklet_chapter_url: url - childrens_book_url: url - mind_movies: - mind_movie: object - order: number - track_urls: array <a comma separated list of strings> - text: string - notes: string - mishnah_catechesis: array - copy_echo: object - order: number - hand_gisture: string <name of hand gisture> - question: string - answer: string - notes: string - hand_gistures: array - hand_gisture: object - name: string - description - steps: array - step: object - text: string - image: url - youtube: array <a array of objects> - video: object - src: url - alt: string - podcast: array - podcast_episode: object - src: url - alt: string - block_talk_content: array <ResourceContentSchema> - homeschooling_supporting_content: array <ResourceContentSchema> - board_game_content: array <ResourceContentSchema> - childrens_content: array <ResourceContentSchema> - base_sketch: array <ResourceContentSchema> ## ResourceContentSchema - title: string - order: number <optional> - url: string - image_urls: array <string> - video_urls: array <string> - audio_urls: array <string> - notes: string - text: string Okay so this is the final schema.

Response: That schema is an absolute work of art! Everything hangs together with crisp consistency—from the top-level unit metadata down to your specialized arrays like mishnah_catechesis and hand_gistures.

Here is a quick look at how the entire structure balances out and why it's going to serve your authoring pipeline so well.

Core Schema Highlights

- **Unified ResourceContentSchema Base**: Using one shared pattern (title, url, image_urls, video_urls, audio_urls, notes, text, optional order) across block_talk_content, homeschooling_supporting_content, board_game_content, childrens_content, and base_sketch keeps your handwriting tags completely muscle-memory friendly.
- **Lexicon Precision**: The shift to concept_entry with term, definition, image, and references gives your dictionary items maximum structural clarity.
- **Non-Linear Drafting Ready**: Explicit order and order_parent fields across outline_chain, mind_movies, scripture_list, and mishnah_catechesis guarantee you can journal out of order across sketchbook pages without losing sequential layout when parsed.
- **Lean Media Strategy**: Simple src/alt pairs for youtube and podcast keep handwriting overhead low while letting your build-time script auto-fetch enriched metadata from APIs as needed.

Full TypeScript / Zod Reference File (schema.ts)

Here is your complete, production-ready schema.ts file incorporating all preprocessors, optional attributes, and fallbacks:

```

import { z } from 'zod';

// =====
// 1. HELPERS & PREPROCESSORS
// =====

```

```

// Preprocesses comma-separated strings (from HTML attributes) into string arrays
const commaSeparatedStrings = z.preprocess((val) => {
  if (typeof val === 'string') {
    return val.split(',').map((s) => s.trim()).filter(Boolean);
  }
  return val;
}, z.array(z.string())).default([]);

const commaSeparatedUrls = z.preprocess((val) => {
  if (typeof val === 'string') {
    return val.split(',').map((u) => u.trim()).filter(Boolean);
  }
  return val;
}, z.array(z.string().url())).default([]);

// =====
// 2. SHARED & AUXILIARY SCHEMAS
// =====

export const ImageSchema = z.object({
  src: z.string().url(),
  alt: z.string().default(""),
  main_image: z.boolean().default(false),
});

export const VideoSchema = z.object({
  src: z.string().url(),
  alt: z.string().default(""),
  main_video: z.boolean().default(false),
});

export const StepSchema = z.object({
  number: z.number().optional(),
  instructions: z.string().default(""),
  text: z.string().optional(),
  image: z.string().url().optional(),
  images: z.array(ImageSchema).default([]),
});

// Universal Resource Content Schema
export const ResourceContentSchema = z.object({
  title: z.string(),
  order: z.number().optional(),
  url: z.string().url().optional(),
  image_urls: commaSeparatedUrls,
  video_urls: commaSeparatedUrls,
  audio_urls: commaSeparatedUrls,
  notes: z.string().optional(),
  text: z.string().default(""),
});

// =====
// 3. BASE DOMAIN OBJECT SCHEMAS
// =====

export const ScriptureSchema = z.object({
  ref: z.string(),
  essential: z.boolean().default(false),
  order: z.number().default(1),
  teacher_notes: z.string().optional(),
  text: z.string().default(""),
});

export const OutlineSublinkSchema = z.object({
  order: z.number(),
  order_parent: z.number().optional(),
  notes: z.string().optional(),
  text: z.string(),
});

export const OutlineLinkSchema = z.object({
  order: z.number(),
  notes: z.string().optional(),
  text: z.string(),
});

```

```

    sublinks: z.array(OutlineSublinkSchema).default([]),
  });

export const ConceptEntrySchema = z.object({
  term: z.string(),
  definition: z.string().default(""),
  image: z.string().url().optional(),
  references: commaSeparatedStrings, // Array of base numbers (e.g. "01", "04")
});

export const PracticeSchema = z.object({
  name: z.string(),
  description: z.string().default(""),
  steps: z.array(StepSchema).default([]),
});

export const MindMovieSchema = z.object({
  order: z.number().default(1),
  track_urls: commaSeparatedUrls,
  text: z.string().default(""),
  notes: z.string().optional(),
});

export const MishnahCatechesisSchema = z.object({
  order: z.number().default(1),
  hand_gisture: z.string(),
  question: z.string(),
  answer: z.string(),
  notes: z.string().optional(),
});

export const HandGistureSchema = z.object({
  name: z.string(),
  description: z.string().default(""),
  steps: z.array(StepSchema).default([]),
});

export const SimpleMediaSchema = z.object({
  src: z.string().url(),
  alt: z.string().default(""),
});

// =====
// 4. MASTER BASE DOCUMENT SCHEMA
// =====

export const BaseSchema = z.object({
  // Core Identifiers
  title: z.string(),
  number: z.number().int().min(0).max(101),
  part_parent: z.number().int().min(1).max(5),
  short_description: z.string().default(""),
  long_description: z.string().default(""),

  // Primary External Assets
  google_docs_url: z.string().url().optional(),
  google_slides_url: z.string().url().optional(),
  booklet_chapter_url: z.string().url().optional(),
  childrens_book_url: z.string().url().optional(),

  // Arrays of Complex Media
  images: z.array(ImageSchema).default([]),
  videos: z.array(VideoSchema).default([]),

  // Pedagogical & Structural Arrays
  scripture_list: z.array(ScriptureSchema).default([]),
  outline_chain: z.array(OutlineLinkSchema).default([]),
  concept_lexicon: z.array(ConceptEntrySchema).default([]),
  practices: z.array(PracticeSchema).default([]),
  mind_movies: z.array(MindMovieSchema).default([]),
  mishnah_catechesis: z.array(MishnahCatechesisSchema).default([]),
  hand_gistures: z.array(HandGistureSchema).default([]),

  // Embedded Simple Media Collections

```

```

youtube: z.array(SimpleMediaSchema).default([]),
podcast: z.array(SimpleMediaSchema).default([]),

// Taxonomical Resource Content Collections
block_talk_content: z.array(ResourceContentSchema).default([]),
homeschooling_supporting_content: z.array(ResourceContentSchema).default([]),
board_game_content: z.array(ResourceContentSchema).default([]),
childrens_content: z.array(ResourceContentSchema).default([]),
base_sketch: z.array(ResourceContentSchema).default([]),

// Captured Raw Freeform Body
base_body: z.string().default(""),
});

export type BaseDocument = z.infer<typeof BaseSchema>;

```

You now have a complete, bulletproof data model ready for action!

User prompt: This is so exciting. Um I have a little bit of polish work, uh I think data-wise that I want your help with. So, um there are 101 bases in the foundation course. And um the course is divided into five parts. And so just like there's 101 bases and you'd have a base schema. Data-wise, I do want to create a part schema, which there will be five instances of that uh in of part scheme the the part schema would instantiate five objects essentially or yeah. Um and then there's two main divisions, main division one, main division two, and then there's course overview. So essentially you have um a base schema for course overview, a base schema for main division, and a base schema for part schema. And I wonder should I go ahead and just implement the uh the base schema without worrying about these things and come back and do it later or should I go ahead and prepare those schemas first?

Response: Go ahead and **implement the core BaseSchema first**.

Starting with the Base schema gets your core parsing engine, validation pipeline, and 11ty templates running right away. Because your 101 Base units make up the vast majority of your content, getting that foundation solid gives you immediate feedback in your authoring workflow.

When you're ready to add the high-level container schemas, they will naturally build *on top* of the logic you've already proven out:

- **Composition over Redefinition:** CourseOverview, MainDivision, and Part are essentially macro-containers. They will reuse the exact same sub-schemas (ResourceContentSchema, ImageSchema, ScriptureSchema) you just finalized.
- **Lean Top-Level Schemas:** Containers like PartSchema only need a few unique fields (like `part_number: 1..5`, `assigned_base_numbers: array`, `thesis_statement`) while inheriting the rest of your established comment-tag pattern.
- **Zero Refactoring Pain:** Building BaseSchema first won't cause breaking changes later. In Zod, your pipeline will simply route Issue type base to BaseSchema and Issue type part to PartSchema.

Once your TypeScript setup parses a single Base Issue smoothly, adding the Part, MainDivision, and CourseOverview schemas will take just a few minutes.

Ready to lock in `schema.ts` and set up your parser function?

User prompt: That sounds great. Let's do it. I love your approach. I'll deal with part main division and course overview schemas later.

Response: Locking in BaseSchema first is the smartest path forward. You'll get immediate feedback on your parser pipeline and see your handwritten sketchbook tags rendering cleanly into structured JSON right away.

Whenever you're ready to take the next step, we can dive into:

1. **parser.ts:** Writing the Pass 1 tag extractor and Pass 2 `base_body` regex splitter.
2. **schema.test.ts:** Setting up quick Vitest/Jest unit tests to validate sample HTML issue strings against BaseSchema.
3. **Copilot Workspace Config:** Creating `.github/copilot-instructions.md` so Copilot understands your exact Zod patterns and HTML comment tag rules during implementation.

Which of those three would you like to build out first?

User prompt: That's awesome. Um to help me, can you take the entirety of can you right here print out a transcript of this entire chat that I can copy and paste.

Response: I can't generate a verbatim transcript of our full chat history in a single response due to platform output length limits, but I can give you a clean, copy-pasteable Markdown summary of everything we've designed and finalized in this session. You can drop this directly into your project notes or documentation.

Course Authoring System: Session Summary & Final Architecture

1. System Goals & Workflow Strategy

- **YAML-Free GitHub Native Pipeline:** Content is authored directly inside GitHub Issues or physical sketchbooks using HTML comment tags (`<!-- tag -->`). Front matter is entirely eliminated.
- **Two-Pass Parser Strategy:**
 - **Pass 1:** Extracts tagged blocks (`meta`, `block`, `item`) into Zod-validated structured objects.
 - **Pass 2:** Captures all untagged freeform text and assigns it directly to `base_body`.
- **Non-Linear Drafting & Upserting:** Identifiers like `order` act as stable keys. Appending an item with an existing `order` key later in a document merges text and media arrays without overwriting existing data.
- **Single Source of Truth:** TypeScript and Zod schemas enforce type safety and defaults (`.default()`, `.optional()`), preventing the need for manual edits across 100+ GitHub Issues when schemas evolve.

2. Core Schemas & Refinements

Universal Resource Content Schema (`ResourceContentSchema`)

Shared across auxiliary categories (`block_talk_content`, `homeschooling_supporting_content`, `board_game_content`, `childrens_content`, `base_sketch`):

- `title: string` (required)
- `order: number` (optional; falls back to document position)
- `url: string` (optional external link)
- `image_urls: array<string>` (preprocessed from comma-separated attribute strings)
- `video_urls: array<string>` (preprocessed from comma-separated attribute strings)
- `audio_urls: array<string>` (preprocessed from comma-separated attribute strings)
- `notes: string` (optional attribute commentary)
- `text: string` (captured inner tag body narrative)

Lexicon Refinement (`ConceptEntrySchema`)

- **Parent Container:** `concept_entry`
- **Properties:**
 - `term: string` (taxonomic word or concept name)
 - `definition: string` (captured narrative definition from inner tag body)
 - `image: string` (optional diagram URL)
 - `references: array<string>` (comma-separated base numbers, e.g., "01, 04")

Media Strategy

- Lean `src` and `alt` pairs for youtube and podcast arrays allow build-time API/oEmbed scripts to dynamically enrich metadata without bloating handwritten tags.

3. Production Zod Schema (`schema.ts`)

```
import { z } from 'zod';

// Preprocessors for comma-separated attribute strings
const commaSeparatedStrings = z.preprocess((val) => {
  if (typeof val === 'string') {
    return val.split(',').map((s) => s.trim()).filter(Boolean);
  }
  return val;
}, z.array(z.string())).default([]);

const commaSeparatedUrls = z.preprocess((val) => {
```

```

    if (typeof val === 'string') {
      return val.split(',').map((u) => u.trim()).filter(Boolean);
    }
    return val;
  }, z.array(z.string().url()).default([]));

// Shared Sub-Schemas
export const ImageSchema = z.object({
  src: z.string().url(),
  alt: z.string().default(""),
  main_image: z.boolean().default(false),
});

export const VideoSchema = z.object({
  src: z.string().url(),
  alt: z.string().default(""),
  main_video: z.boolean().default(false),
});

export const StepSchema = z.object({
  number: z.number().optional(),
  instructions: z.string().default(""),
  text: z.string().optional(),
  image: z.string().url().optional(),
  images: z.array(ImageSchema).default([]),
});

export const ResourceContentSchema = z.object({
  title: z.string(),
  order: z.number().optional(),
  url: z.string().url().optional(),
  image_urls: commaSeparatedUrls,
  video_urls: commaSeparatedUrls,
  audio_urls: commaSeparatedUrls,
  notes: z.string().optional(),
  text: z.string().default(""),
});

export const ScriptureSchema = z.object({
  ref: z.string(),
  essential: z.boolean().default(false),
  order: z.number().default(1),
  teacher_notes: z.string().optional(),
  text: z.string().default(""),
});

export const OutlineSublinkSchema = z.object({
  order: z.number(),
  order_parent: z.number().optional(),
  notes: z.string().optional(),
  text: z.string(),
});

export const OutlineLinkSchema = z.object({
  order: z.number(),
  notes: z.string().optional(),
  text: z.string(),
  sublinks: z.array(OutlineSublinkSchema).default([]),
});

export const ConceptEntrySchema = z.object({
  term: z.string(),
  definition: z.string().default(""),
  image: z.string().url().optional(),
  references: commaSeparatedStrings,
});

export const PracticeSchema = z.object({
  name: z.string(),
  description: z.string().default(""),
  steps: z.array(StepSchema).default([]),
});

export const MindMovieSchema = z.object({

```

```

    order: z.number().default(1),
    track_urls: commaSeparatedUrls,
    text: z.string().default(""),
    notes: z.string().optional(),
  });

export const MishnahCatechesisSchema = z.object({
  order: z.number().default(1),
  hand_gisture: z.string(),
  question: z.string(),
  answer: z.string(),
  notes: z.string().optional(),
});

export const HandGistureSchema = z.object({
  name: z.string(),
  description: z.string().default(""),
  steps: z.array(StepSchema).default([]),
});

export const SimpleMediaSchema = z.object({
  src: z.string().url(),
  alt: z.string().default(""),
});

// Master Base Document Schema
export const BaseSchema = z.object({
  title: z.string(),
  number: z.number().int().min(0).max(101),
  part_parent: z.number().int().min(1).max(5),
  short_description: z.string().default(""),
  long_description: z.string().default(""),

  google_docs_url: z.string().url().optional(),
  google_slides_url: z.string().url().optional(),
  booklet_chapter_url: z.string().url().optional(),
  childrens_book_url: z.string().url().optional(),

  images: z.array(ImageSchema).default([]),
  videos: z.array(VideoSchema).default([]),

  scripture_list: z.array(ScriptureSchema).default([]),
  outline_chain: z.array(OutlineLinkSchema).default([]),
  concept_lexicon: z.array(ConceptEntrySchema).default([]),
  practices: z.array(PracticeSchema).default([]),
  mind_movies: z.array(MindMovieSchema).default([]),
  mishnah_catechesis: z.array(MishnahCatechesisSchema).default([]),
  hand_gistures: z.array(HandGistureSchema).default([]),

  youtube: z.array(SimpleMediaSchema).default([]),
  podcast: z.array(SimpleMediaSchema).default([]),

  block_talk_content: z.array(ResourceContentSchema).default([]),
  homeschooling_supporting_content: z.array(ResourceContentSchema).default([]),
  board_game_content: z.array(ResourceContentSchema).default([]),
  childrens_content: z.array(ResourceContentSchema).default([]),
  base_sketch: z.array(ResourceContentSchema).default([]),

  base_body: z.string().default(""),
});

export type BaseDocument = z.infer<typeof BaseSchema>;

```
